



ACADÉMIE
DE NANTES

Liberté
Égalité
Fraternité

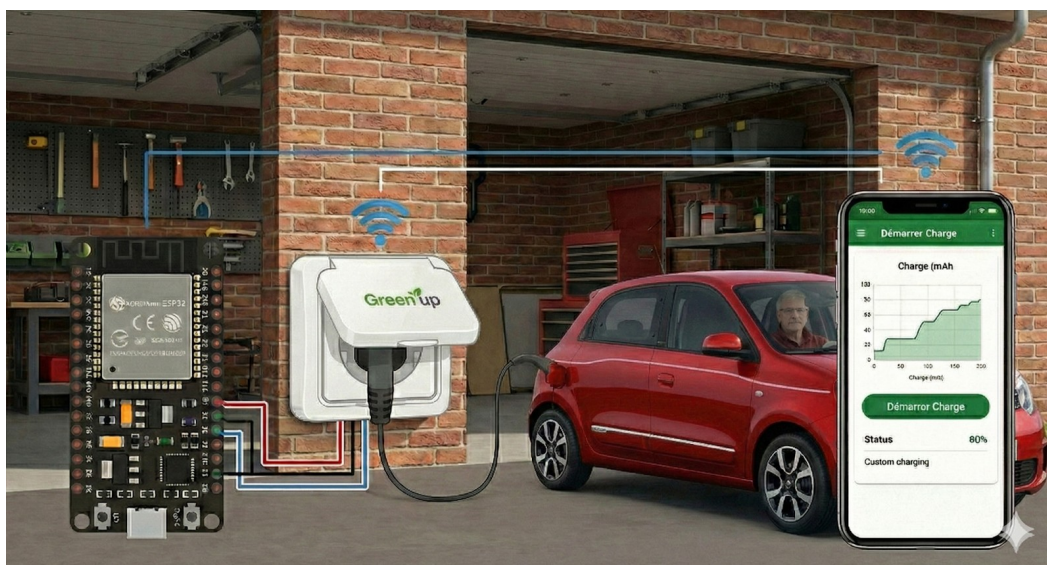


Lycée
TOUCHARD
WASHINGTON

Dossier Personnel

BTS CIEL SESSION 2025 – IR E6 – PROJET TECHNIQUE

Gestionnaire de charge de véhicules électrique



Lycée : <u>Touchard-Washington</u>		Ville : LE MANS
Nom du projet :	Gestionnaire de charge de Véhicules électriques	
N° du projet :	TW2	

Projet nouveau	Oui ☐ Non ☐	Projet interne	Oui ☒ Non ☐
Délai de réalisation	150 heures	Statut des étudiants	Formation initiale ☒ Apprentissage ☐
Spécialité des étudiants	ER ☐ IR ☒ Mixte ☐	Nombre d'étudiants :	4 étudiants
Professeurs responsables	<u>Philippe CRUCHET</u> , Didier Bernard, François Martin, Anthony LE <u>CREN</u> , Jilali <u>KHAMACH</u> , François <u>QUEREC</u>		

Table des matières

1 Cas D'utilisation.....	4
1.1 Gérer les utilisateurs.....	4
1.2 IHM.....	5
2 Superviser plusieurs bornes.....	9
2.1 IHM.....	10
2.2 Diagramme de séquences.....	12
3 Paramétrer à distance le système.....	13
3.1 IHM.....	14
3.2 Diagramme de séquence.....	15
4 Consulter l'historique de la charge.....	16
4.1 IHM.....	17
4.2 Diagramme de séquence.....	18
5 Exporter les données.....	19
5.1 IHM.....	20
5.2 Diagramme de séquence.....	21
6 Choix matériel.....	22
7 Choix Logiciel.....	24
7.1 Choix structurelle.....	25
7.2 PHP.....	26
Rôle de PHP 8 dans la sécurité et la gestion de la logique applicative :.....	26
7.3 MariaDB.....	27
8 Diagramme de classes.....	32
Vue (le front-end JS).....	32
Contrôleur (contrôleur.php).....	32
modele_utilisateur.....	33
modele_bornes.....	33
modele_mesure.....	33
9 Fiche de test.....	34
10 Diagramme de Gantt.....	36
11 Compétences acquises.....	37
1. Maîtrise des Architectures Logicielles et du Développement Back-End.....	37
2. Optimisation des Interfaces Évolué (Front-End) et UX.....	37
3. Autonomie et Rigueur Méthodologique.....	37
12 Perspectives d'améliorations.....	38
Actualisation automatique du graphique sans recharger (Temps Réel).....	38
2. Création d'un tableau de bord général (Dashboard).....	38
3. Système de gestion des utilisateurs et des droits (Profils).....	38

1 Cas D'utilisation

1.1 Gérer les utilisateurs

Pré conditions	Base de données accessible. Administrateur connecté (<code>\$_SESSION['connecte'] === true</code>).
Post conditions	La table UTILISATEURS est mise à jour selon l'action choisie (modification, suppression ou basculement d'état). Le tableau de l'interface graphique (DataTables) est actualisé de manière dynamique.
Scénario principal	L'administrateur accède au tableau de gestion des utilisateurs (<code>admin.php</code>). - Le système charge et affiche la liste des utilisateurs dans un tableau structuré (Identifiant caché, Login, État sous forme de badge, et Actions disponibles). - L'administrateur sélectionne une action parmi les choix suivants : - Modifier le mot de passe, Supprimer un utilisateur, Activer / Suspendre un compte - Le système exécute l'action demandée : - Si Modification : L'administrateur saisit le nouveau mot de passe dans une fenêtre modale. Le système chiffre le mot de passe (password_hash) et met à jour la base de données (commande ModifierUtilisateur). - Si Suppression : L'administrateur valide la confirmation dans une fenêtre modale. Le système retire l'entrée de la base de données (commande SupprimerUtilisateur). - Si Changement d'état : L'administrateur bascule l'interrupteur (<i>switch</i>). Le système modifie le champ <code>etat</code> (valeur 1 ou 0) en base de données (commandes ActiverUtilisateur ou SuspendreUtilisateur). - Le système confirme la réussite de l'action en fermant la fenêtre modale concernée ou en mettant à jour dynamiquement la ligne du tableau (DataTables) sans recharger la page.
Alternatives	A1 – Modification : champ mot de passe vide : message d'erreur « Erreur lors de la modification » s'affiche dans la modale. A2 – Erreur BDD (ModifierUtilisateur / SupprimerUtilisateur / ChangerEtatUtilisateur) : le modèle retourne 'erreur' ; le contrôleur répond {statut: 'erreur'} ; le front affiche un message d'erreur dans la modale (Modifier) ou ne ferme pas la modale (Supprimer).

1.2 IHM

Gestion des Utilisateurs
Administration des comptes de la plateforme

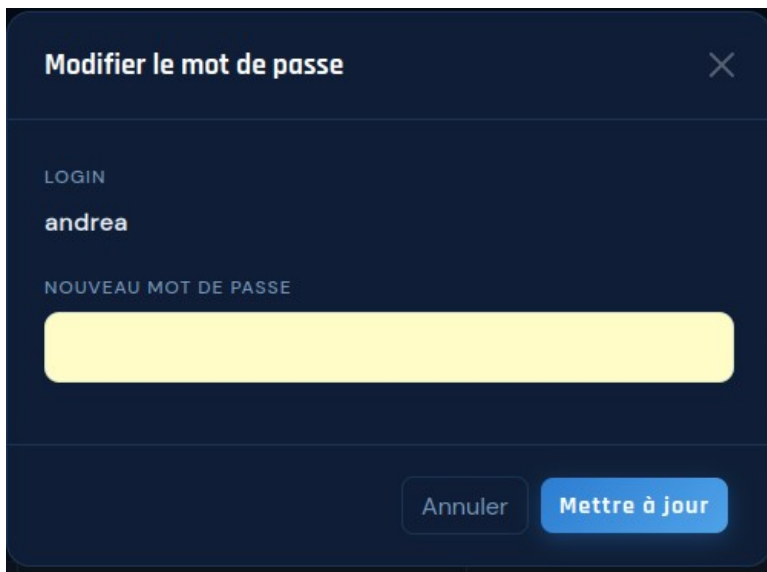
Show **10** entries Search:

LOGIN	ÉTAT	ACTIONS
andrea	Actif	  
admin	Actif	  
valou	Actif	  
alexys	Actif	  
camille	Actif	  
nael	Actif	  

Showing 1 to 6 of 6 entries

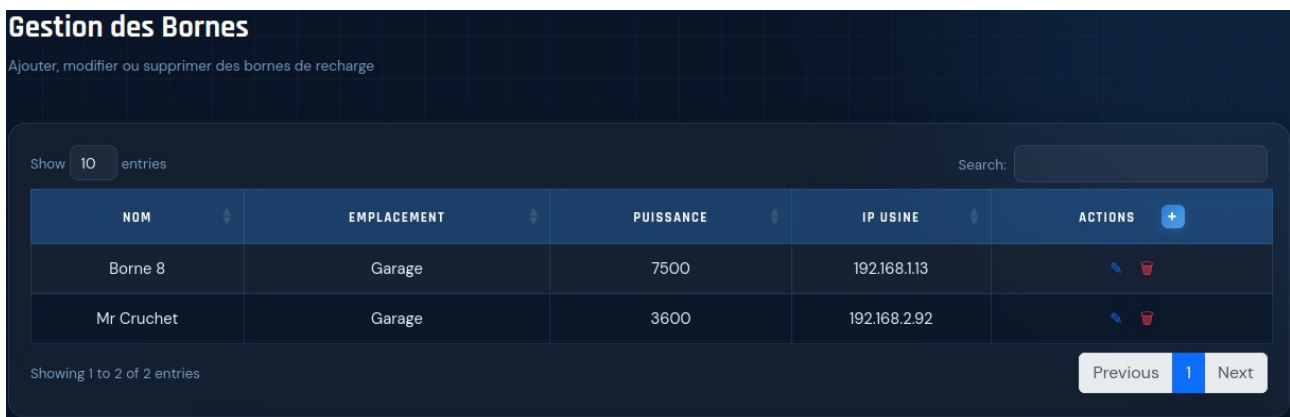
Previous **1** Next

Figure 1: Interface principale de gestion des utilisateurs



The modal is titled "Modifier le mot de passe" with a close button (X) in the top right corner. It contains two input fields: "LOGIN" with the value "andrea" and "NOUVEAU MOT DE PASSE" which is currently empty. At the bottom, there are two buttons: "Annuler" (Cancel) and "Mettre à jour" (Update).

Figure 2: Interface remplacer le mot de passe d'un utilisateur



The interface is titled "Gestion des Bornes" with a subtitle "Ajouter, modifier ou supprimer des bornes de recharge". It features a table with 5 columns: NOM, EMPLACEMENT, PUISSANCE, IP USINE, and ACTIONS. There are 2 rows of data. Below the table, it says "Showing 1 to 2 of 2 entries". At the bottom right, there are navigation buttons: "Previous", "1" (selected), and "Next".

NOM	EMPLACEMENT	PUISSANCE	IP USINE	ACTIONS
Borne 8	Garage	7500	192.168.1.13	 
Mr Cruchet	Garage	3600	192.168.2.92	 

Figure 3: Interface gestion des bornes



Figure 4: Interface supprimer un utilisateur

1.3 Diagramme de séquences

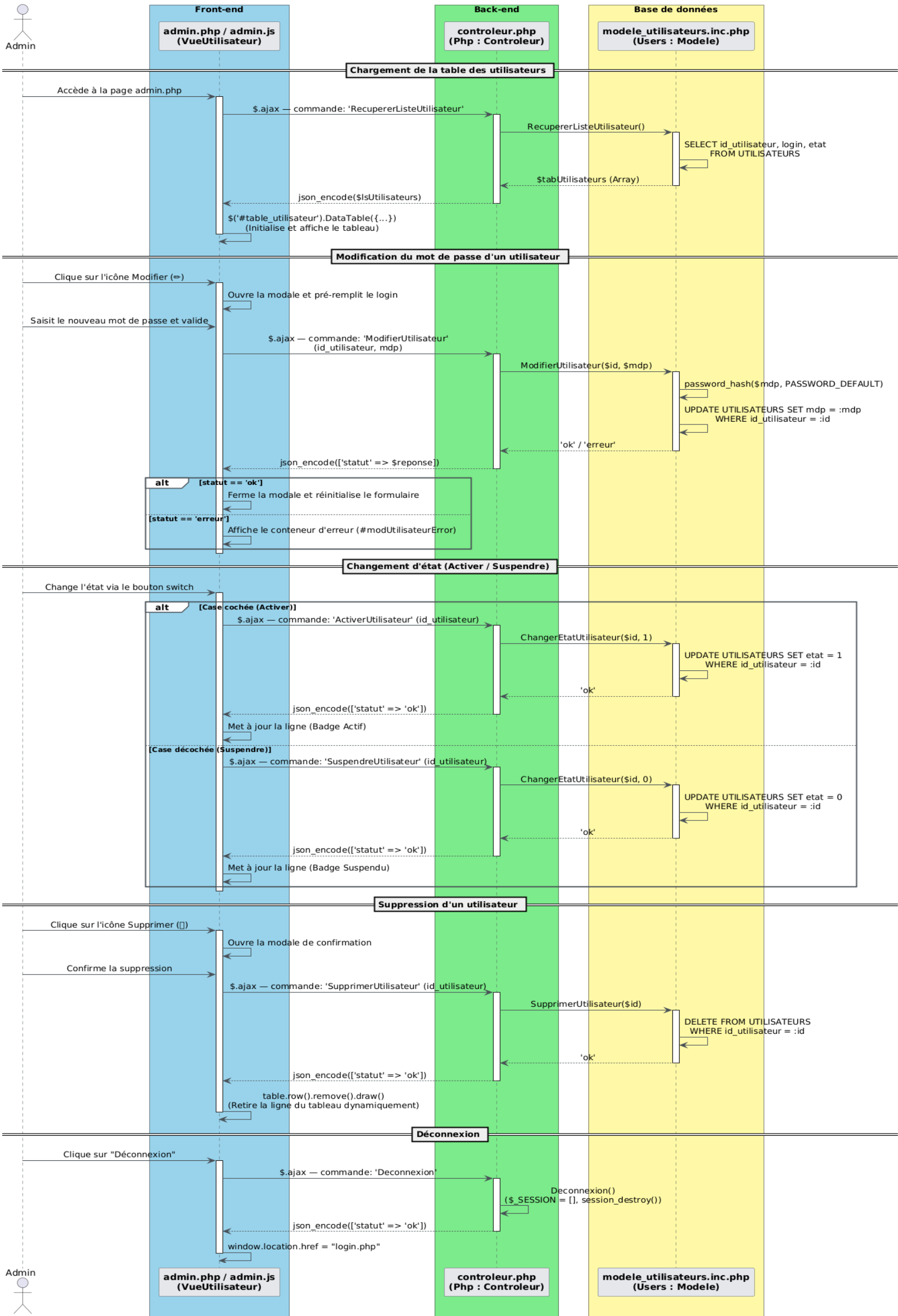


Figure 5: Diagramme de séquence du cas "Gerer utilisateurs"

2 Superviser plusieurs bornes

Pré conditions	L'administrateur doit être connecté de manière valide (\$_SESSION['connecte'] === true)
Post conditions	La table BORNES de la base de données est mise à jour (insertion, modification ou suppression d'une ligne).
Scénario principal	Affichage de la liste : • L'administrateur ouvre l'interface de gestion. • Le système récupère et affiche la liste des bornes (Nom, Emplacement, Puissance, IP usine). Ajout d'une borne : • L'administrateur clique sur (+), saisit les informations de la borne dans la modale et valide. • Le système enregistre la borne et actualise le tableau. Modification d'une borne : • L'administrateur clique sur le bouton modifier et modifie les champs nécessaires et valide. • Le système met à jour la base de données et actualise le tableau. Suppression d'une borne : • L'administrateur clique sur le bouton poubelle et confirme la suppression. • Le système supprime la borne et met à jour le tableau.
Alternatives	Formulaire incomplet : Le JavaScript bloque l'envoi et la modale reste ouverte. Erreur serveur / SQL : Le système affiche un message d'erreur rouge dans la modale. Session non authentifiée : Le système bloque l'accès et redirige vers la page de connexion.

2.1 IHM



Figure 6: Interface gestion des bornes

The screenshot shows the 'Ajouter une borne' (Add a charging station) form. It has a title bar with a close button (X). The form contains four input fields: 'NOM DE LA BORNE', 'EMPLACEMENT', 'PUISSANCE' (with a dropdown arrow), and 'IP USINE'. At the bottom, there are two buttons: 'Annuler' (Cancel) and 'Ajouter' (Add).

Figure 7: Interface ajouter une borne

Modifier une borne ✕

NOM DE LA BORNE

Borne 8

EMPLACEMENT

Garage

PUISSANCE

7500

IP USINE

192.168.1.13

Annuler Mettre à jour

Figure 8: Interface modifier une borne

Supprimer une borne ✕

Êtes-vous sûr de vouloir supprimer la borne Borne 8 ?

Annuler Supprimer

Figure 9: Interface supprimer une borne

2.2 Diagramme de séquences

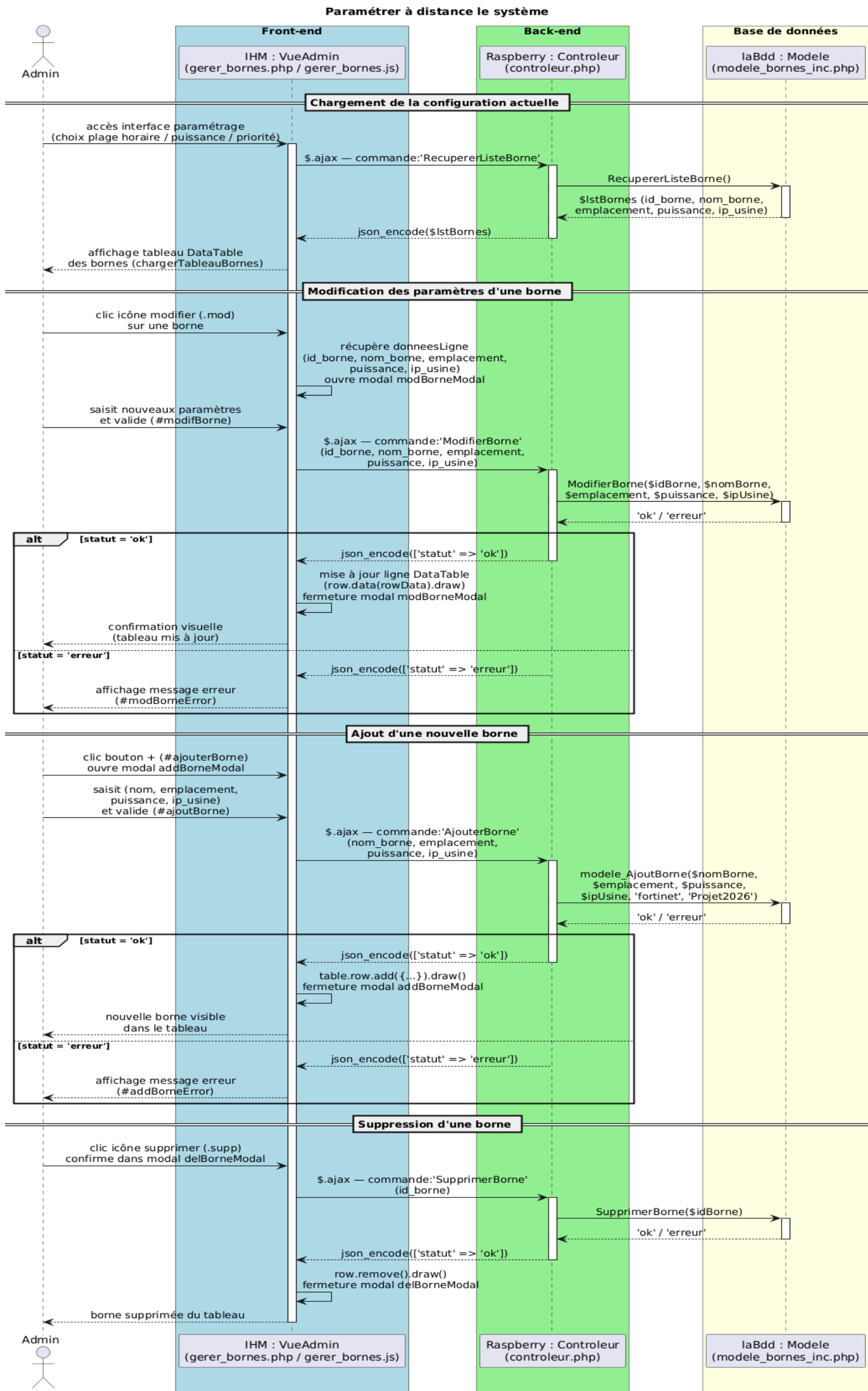


Figure 10: Diagramme de séquence "Paramétrer a distance le systeme"

Vo Van Andrea (Étudiant 4)

Page 11/38

3 Paramétrer à distance le système

Pré conditions	L'utilisateur est authentifié. La base de données est accessible.
Post conditions	//
Scénario principal	L'utilisateur ouvre la page de consultation de la borne (borne.php). • Le système récupère le nom de l'équipement (commande <code>GetNomBorne</code>) et affiche automatiquement le graphique par défaut de la consommation (commande <code>GetMesureParBorne</code>). • L'utilisateur sélectionne une plage de dates (date de début et date de fin) et clique sur "Afficher". • Le système transmet les paramètres via une requête AJAX au contrôleur. • Le système interroge le modèle et met à jour dynamiquement le graphique (<code>Highcharts.chart</code>) à l'écran.
Alternatives	Dates non saisies : • L'utilisateur clique sur "Afficher" en laissant les champs vides. • Le script bloque la validation. • Le système maintient l'affichage actuel du graphique.

3.1 IHM



Figure 11: Interface globale

3.2 Diagramme de séquence

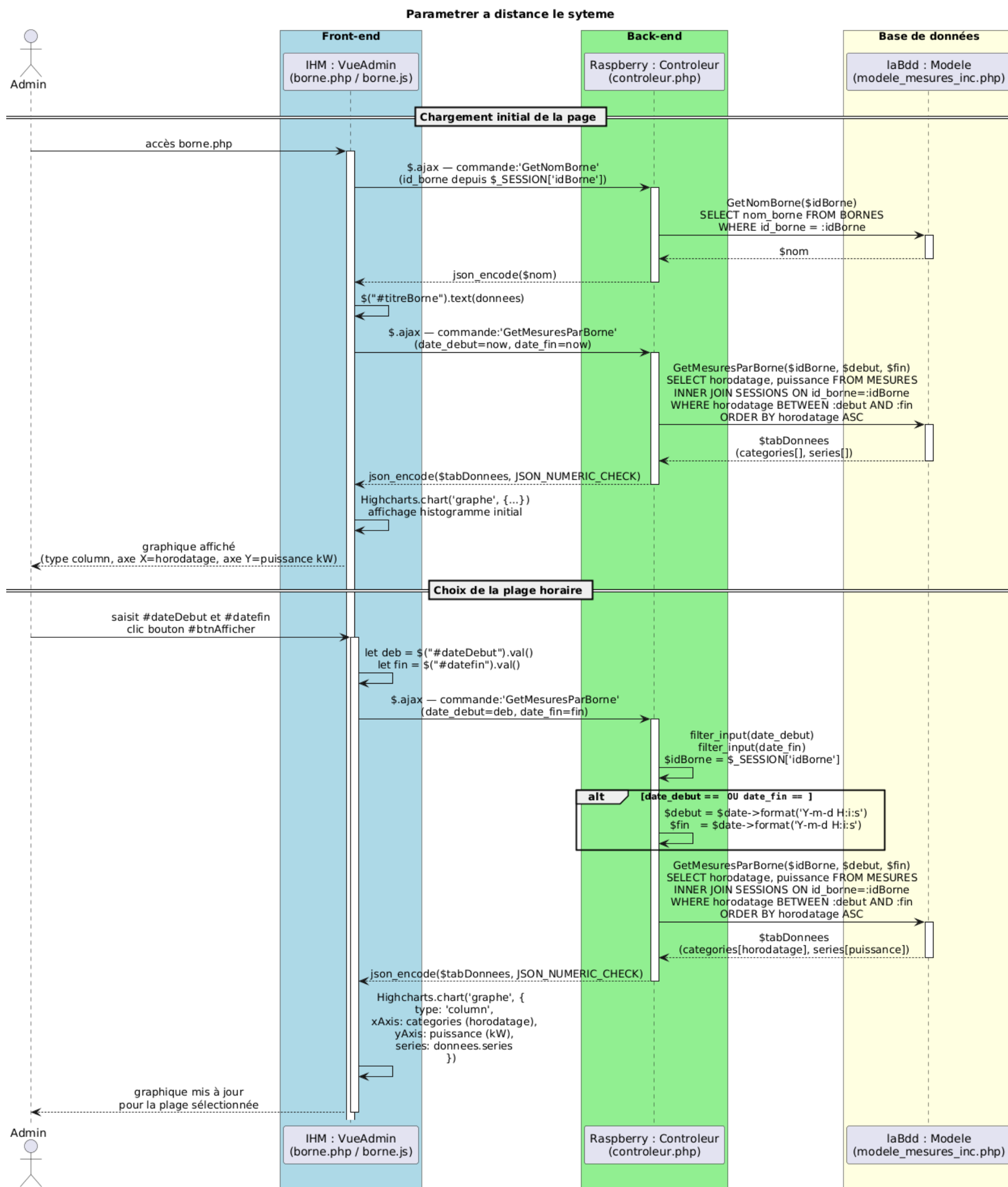
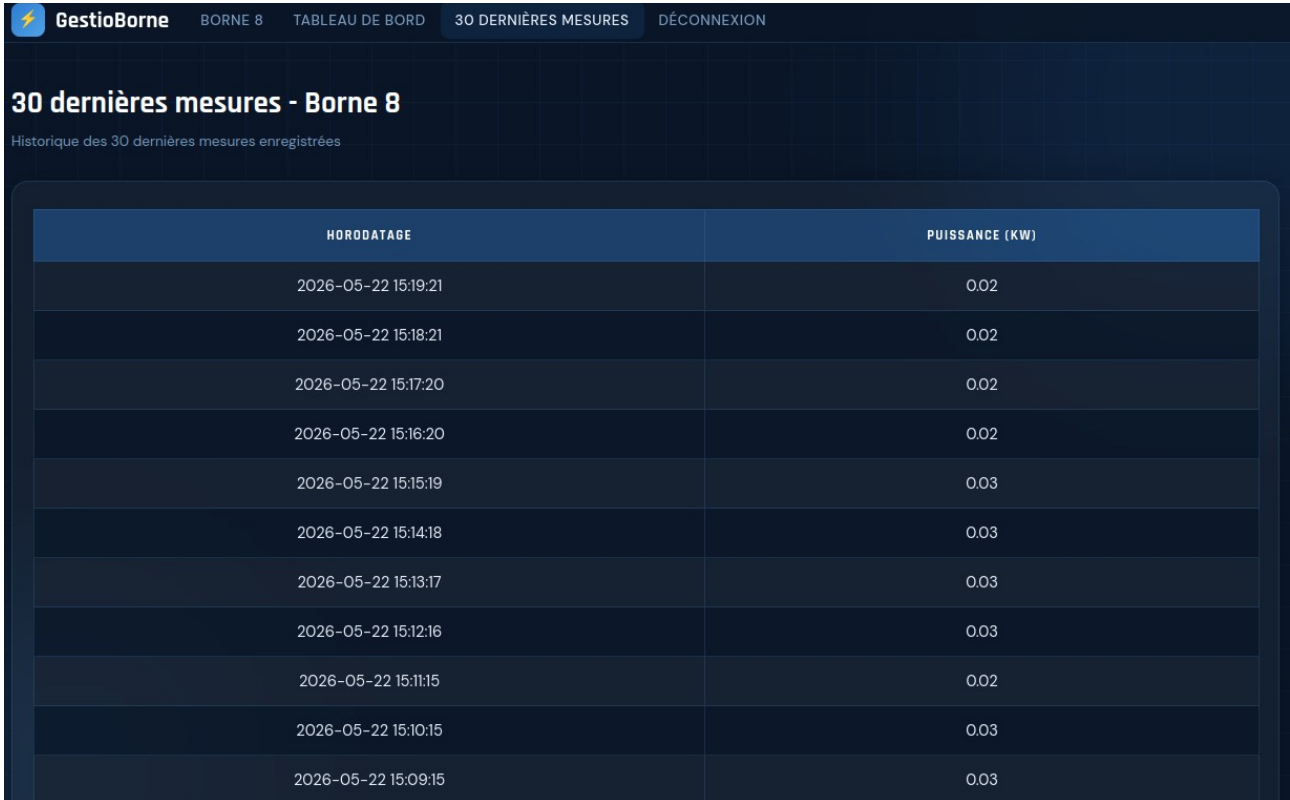


Figure 12: Diagramme de séquence "Parametrer a distance le syteme"

4 Consulter l'historique de la charge

Pré conditions	L'utilisateur est authentifié. La base de données est accessible.
Post conditions	//
Scénario principal	L'utilisateur ouvre l'interface des mesures (<code>mesures.php</code>). - Le système déclenche automatiquement une requête AJAX (commande <code>GetNomBorne</code>) afin de récupérer le nom de la borne ciblée en session et de mettre à jour le titre principal (<code>#titreMesures</code>). - Le système effectue un second appel AJAX en arrière-plan (commande <code>GetDernieresMesuresParBorne</code>). - Le système interroge le modèle (<code>modele_mesures.inc.php</code>) pour exécuter une requête SQL liant les tables <code>MESURES</code> et <code>SESSIONS</code> afin d'extraire les 30 derniers enregistrements de cette borne triés par ordre décroissant. - Le système reçoit les données et construit dynamiquement le tableau interactif (<code>#table_mesures</code> via <code>DataTables</code>) en affichant pour chaque ligne : l'identifiant de la mesure, l'horodatage précis, la puissance en kW et l'identifiant de la session.
Alternatives	A1. Session expirée ou inexistante : - Le système détecte au chargement du fichier PHP que la variable de session n'est pas définie. - Le système bloque l'affichage de l'historique. - L'utilisateur est immédiatement redirigé vers la page <code>login.php</code>.

4.1 IHM



GestioBorne BORNE 8 TABLEAU DE BORD 30 DERNIÈRES MESURES DÉCONNEXION

30 dernières mesures - Borne 8

Historique des 30 dernières mesures enregistrées

HORODATAGE	PUISSANCE (KW)
2026-05-22 15:19:21	0.02
2026-05-22 15:18:21	0.02
2026-05-22 15:17:20	0.02
2026-05-22 15:16:20	0.02
2026-05-22 15:15:19	0.03
2026-05-22 15:14:18	0.03
2026-05-22 15:13:17	0.03
2026-05-22 15:12:16	0.03
2026-05-22 15:11:15	0.02
2026-05-22 15:10:15	0.03
2026-05-22 15:09:15	0.03

Figure 13: Interface de l'historique de la charge

4.2 Diagramme de séquence

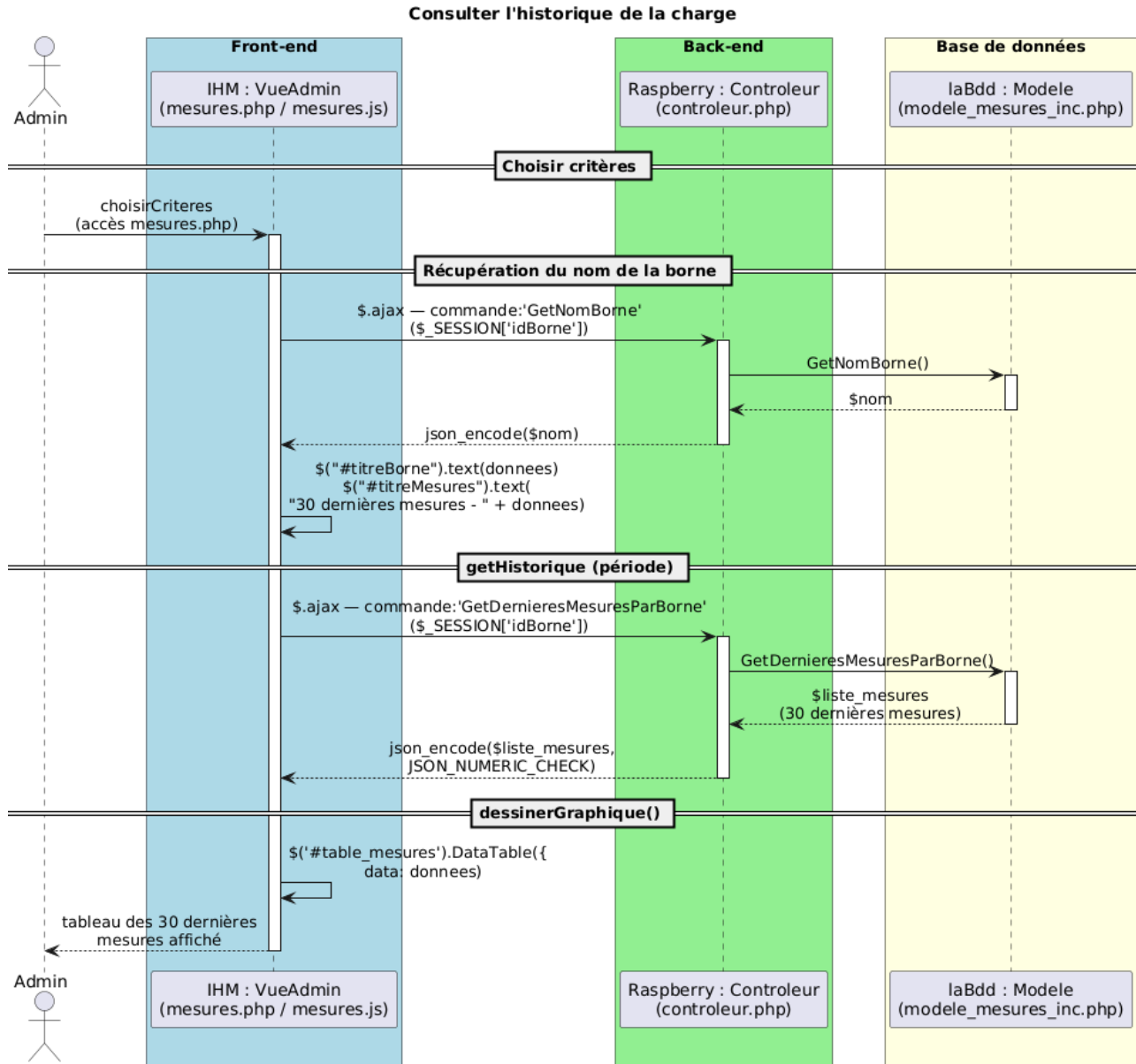


Figure 14: Diagramme de séquence du cas "Consulter l'historique de la charge"

5 Exporter les données

Pré conditions	L'utilisateur est authentifié. La base de données est accessible.
Post conditions	//
Scénario principal	L'utilisateur se trouve sur l'interface de détail de la borne (borne.php) où le graphique de consommation est visible à l'écran. • L'utilisateur clique sur l'icône de menu contextuel (bouton composé de trois lignes horizontales lié au module <code>exporting.js</code>) situé dans le coin supérieur droit du composant graphique. • Le système affiche une liste déroulante proposant deux catégories de formats d'exportation : • Formats images / documents : PNG, JPEG, PDF, SVG. • Formats de données brutes : CSV, XLSX (Excel). • L'utilisateur sélectionne le format souhaité (par exemple : "Download CSV" ou "Download XLSX"). • Le système (via le script Highcharts) intercepte le choix, extrait l'ensemble des coordonnées (horodatages et valeurs de puissance en kW) présentes dans les séries du graphique courant et génère instantanément le fichier associé. • Le système déclenche automatiquement le téléchargement du fichier directement à travers le navigateur web de l'utilisateur.
Alternatives	A1. Graphique vide de données : 1. L'utilisateur clique sur le bouton de menu contextuel alors qu'aucune mesure n'est affichée sur la période sélectionnée. 2. Le système génère un document vierge ou un fichier tableur ne contenant que les en-têtes de colonnes (Horodatage / Puissance).

5.1 IHM

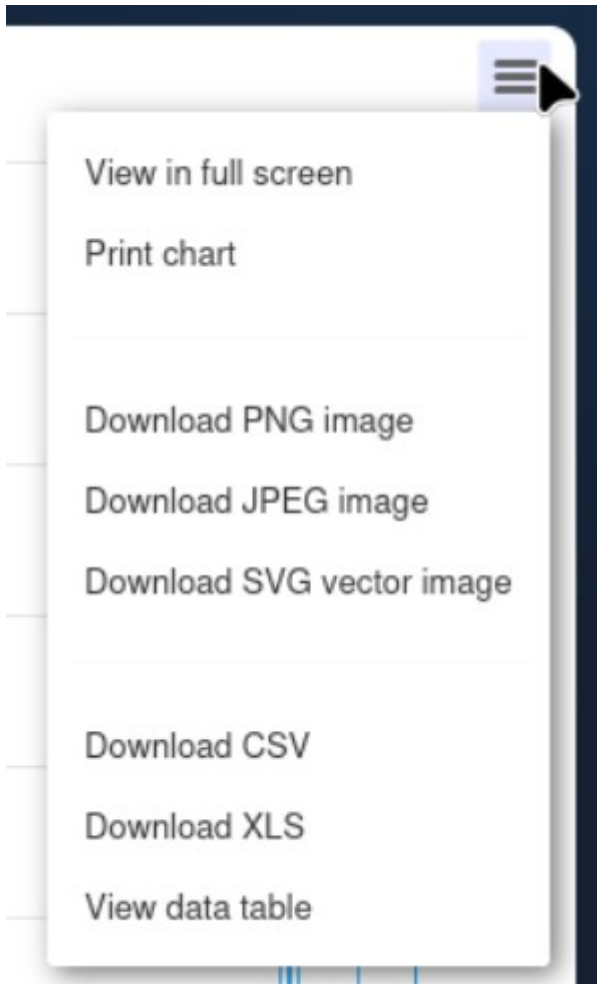
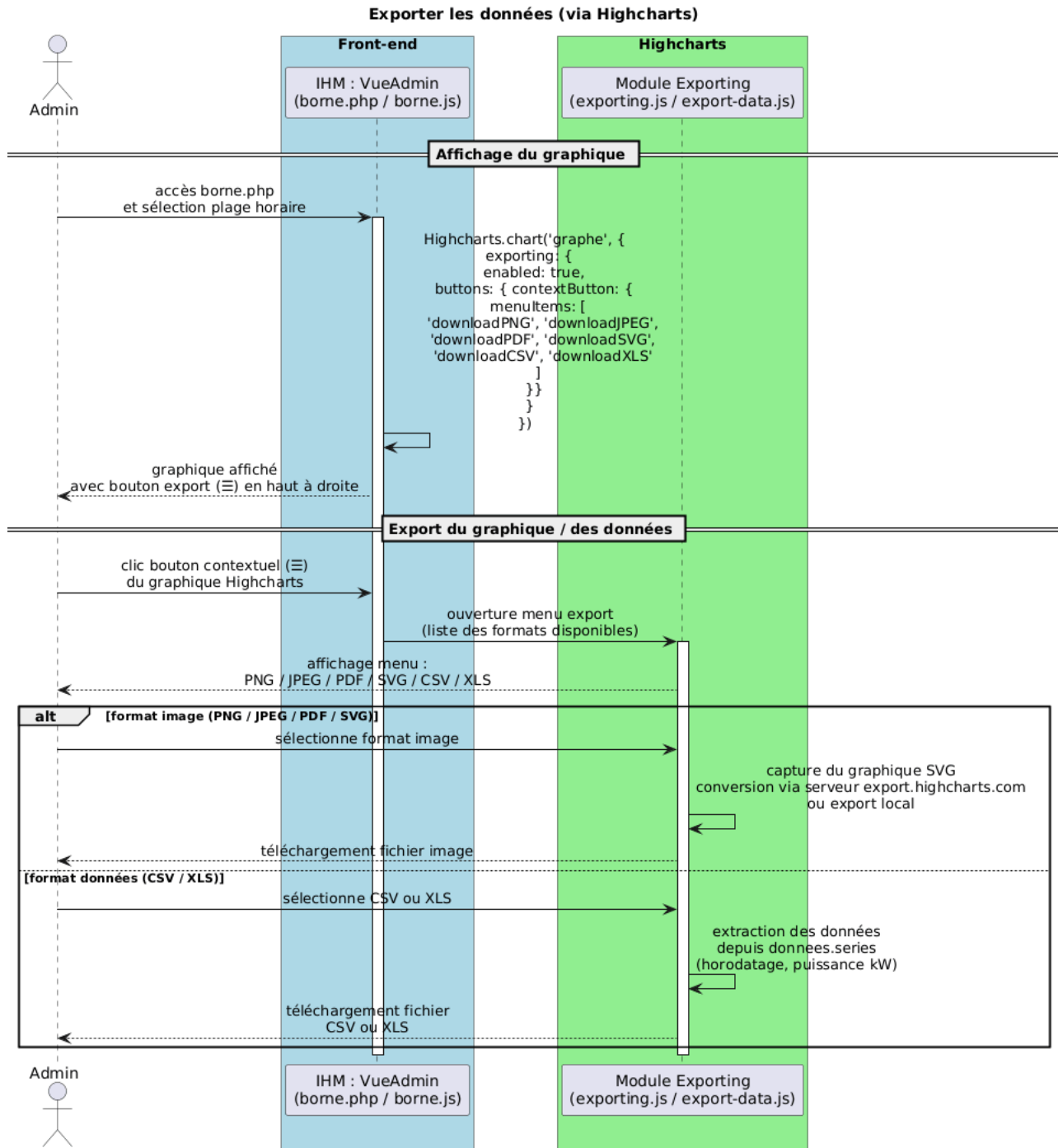


Figure 15: Interface selection format

5.2 Diagramme de séquence



6 Choix matériel



Raspberry Pi 4 modele B

Le cœur applicatif du système repose sur un nano-ordinateur Raspberry Pi 4 Model B. Justification des performances : Équipé d'un processeur ARM Cortex-A72 Quad-Core cadencé à 1,5 GHz, le Raspberry Pi 4 offre la puissance de calcul nécessaire pour faire tourner simultanément un serveur web (Apache/NGINX), un interpréteur de scripts (PHP) et un système de gestion de base de données relationnelle (MariaDB), tout en assurant le traitement des requêtes AJAX asynchrones envoyées par l'interface utilisateur. Gestion de la mémoire vive (RAM) : Le choix de la version dotée de 4 Go de RAM se justifie par la nécessité de maintenir une fluidité optimale lors des accès concurrents sur la base de données, notamment lors de l'extraction et de l'affichage dynamique de gros volumes de données (historique des 30 dernières mesures via DataTables ou rendus graphiques complexes via la bibliothèque Highcharts). Connectivité et réseau : Le Raspberry Pi 4 intègre nativement un contrôleur Gigabit Ethernet ainsi qu'un module Wi-Fi double bande (2.4 GHz / 5 GHz). Cette double connectivité est indispensable dans notre architecture pour assurer la communication IP à distance avec les différentes bornes de recharge du réseau.



Disque Dur 230go SSD Philips

Contrairement aux architectures standards basées sur une carte MicroSD, le stockage du système d'exploitation (Raspberry Pi OS), de l'application web et de la base de données locale a été confié à un disque dur SSD de 250 Go, connecté sur l'un des ports USB 3.0 (bleus) du Raspberry Pi 4. Fiabilité et cycle de vie (Tolérance aux écritures) : Les cartes MicroSD ne sont pas conçues pour supporter des cycles d'écriture intensifs et continus. L'insertion en tâche de fond des mesures de puissance en temps réel (via les scripts liés à la table MESURES) et la génération de logs d'horodatage provoquent une usure prématurée des cellules de mémoire flash d'une carte SD (phénomène de corruption de fichiers). L'utilisation d'un SSD, équipé d'un contrôleur de gestion d'usure performant, sécurise l'intégrité des données à long terme. Performances et taux de transfert : Connecté en USB 3.0, le SSD offre des vitesses de lecture et d'écriture aléatoires nettement supérieures à une carte mémoire de classe 10. Cela réduit drastiquement les temps de latence lors de l'exécution de requêtes SQL complexes (jointures entre les tables MESURES et SESSIONS) requises pour rafraîchir l'interface utilisateur ou générer des historiques. Capacité de stockage : Avec 250 Go, le système dispose d'une marge confortable pour archiver plusieurs années de mesures de consommation d'un parc de bornes complet sans risquer la saturation du disque.

7 Choix Logiciel

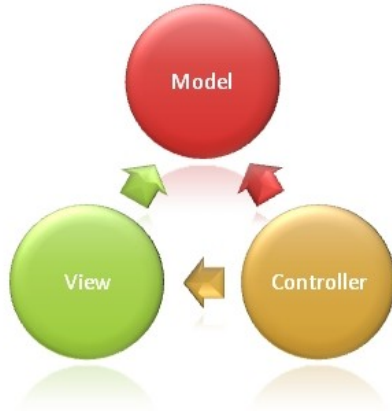


NetBeans

L'environnement de développement intégré (IDE) NetBeans permet de centraliser la gestion du code source et d'optimiser le flux de travail. Pour l'administration du système, une configuration d'accès distant (Remote SSH) lie directement la station de travail au Raspberry Pi 4. Cette architecture repose sur la création d'une plateforme distante au sein de l'IDE, configurée avec l'adresse IP du nano-ordinateur, le port SSH réglementaire et les identifiants d'accès sécurisés. Grâce à ce canal, NetBeans synchronise le répertoire local du projet avec le répertoire du serveur web situé sur le disque SSD du Raspberry Pi.

Cette infrastructure offre une intégration transparente lors des phases de développement et de déploiement. À chaque sauvegarde d'un fichier PHP ou JavaScript, NetBeans exécute un transfert automatique des modifications en arrière-plan via le protocole sécurisé SFTP. Ce mécanisme élimine le besoin de manipulations manuelles ou de transferts de fichiers tiers, permettant d'exécuter, de tester et de déboguer l'application en temps réel directement dans l'environnement de production de la cible.

7.1 Choix structurelle



L'architecture **MVC 2** (Modèle-Vue-Contrôleur de type 2) est un motif de conception (*design pattern*) particulièrement adapté aux applications web dynamiques. Contrairement au MVC 1 où la Vue gère une partie de la logique de contrôle, le MVC 2 introduit un point d'entrée unique : le **Contrôleur**.

Dans un environnement web en MVC 2, les responsabilités sont strictement séparées en trois couches distinctes, orchestrées par le serveur web :

Le Contrôleur (`contrôleur.php`) : C'est la porte d'entrée unique de toutes les requêtes de l'application (notamment vos appels AJAX). Il reçoit la demande de l'utilisateur (par exemple, la commande `GetDernieresMesuresParBorne`), analyse les paramètres, interroge le Modèle adéquat, puis retourne la réponse ou redirige vers la bonne Vue. Aucun code HTML ou SQL ne se trouve ici.

Le Modèle (`modele_bornes.inc.php`, `modele_mesures.inc.php`) : Il représente la couche d'accès aux données et la logique métier. Il contient exclusivement les requêtes SQL (via PDO) pour interagir avec la base de données MariaDB installée sur le SSD de votre Raspberry Pi. Il récupère les données brutes et les renvoie au Contrôleur, sans se soucier de la façon dont elles seront affichées.

La Vue (`borne.php`, `mesures.php` + les scripts `borne.js`, `mesures.js`) : C'est l'interface graphique présentée à l'utilisateur. Elle est responsable de la mise en forme. En MVC 2, la vue est dite "passive" : elle se contente d'afficher les structures (tableaux DataTables, graphiques Highcharts) et d'envoyer les actions de l'utilisateur au Contrôleur via JavaScript/AJAX.

7.2 PHP



Dans ce projet, PHP 8 est utilisé comme langage de

script côté serveur (back-end) installé sur le Raspberry Pi 4. Il constitue le moteur logique central de l'application de supervision, assurant l'interconnexion entre les requêtes asynchrones des interfaces utilisateurs et la base de données MariaDB. PHP 8 prend en charge la validation des privilèges, le traitement des variables d'état et la structuration des flux de données avant leur transmission au client.

Rôle de PHP 8 dans la sécurité et la gestion de la logique applicative :

- **Contrôle de l'authentification (Gestion des Sessions) :** PHP 8 intercepte l'accès à chaque page via l'instruction `session_start()`. Dans des fichiers comme `borne.php` ou `mesures.php`, il valide la persistance de la connexion à l'aide de la superglobale `$_SESSION['connecte']`. Si cette variable est absente ou invalide, PHP bloque instantanément la génération du code HTML et force une redirection vers l'interface `login.php` via la fonction `header()`.
- **Gestion dynamique des privilèges d'administration :** Le serveur exploite les capacités de traitement conditionnel de PHP pour adapter l'interface utilisateur selon le rôle de la session. Dans le fichier `borne.php`, la condition `if ($isAdmin)` évalue si l'utilisateur possède le rôle d'administrateur, déclenchant l'injection ou l'exclusion du code HTML lié à l'onglet d'administration (`admin.php`).
- **Pivot du motif d'architecture MVC 2 :** PHP 8 sert d'aiguilleur exclusif entre l'IHM et la base de données. Il réceptionne les commandes transmises par les requêtes AJAX (telles que `GetNomBorne` ou `GetDernieresMesuresParBorne`), appelle les fonctions correspondantes encapsulées dans les modèles (ex: `modele_mesures.inc.php`), extrait les tableaux de données brutes et les convertit de manière sécurisée en chaînes standardisées.

7.3 MariaDB

MariaDB est un Système de Gestion de Base de Données Relationnel open source, dérivé de MySQL et entièrement compatible avec le langage SQL. Reconnu pour sa stabilité, sa performance et sa légèreté, il convient parfaitement à une utilisation embarquée ou serveur dans des projets de type IoT, comme notre système de borne de recharge.



Dans ce projet, MariaDB est installé sur la Raspberry Pi pour centraliser et historiser l'ensemble des informations du système. Ce stockage structuré rend les données accessibles et exploitables en temps réel par le serveur Qt ainsi que par l'application mobile.

MariaDB est utilisé pour :

- **Stocker durablement les mesures** : Enregistrer les valeurs de puissance moyenne calculées par l'ESP32.
- **Historiser la sécurité de la borne** : Archiver de manière chronologique chaque alerte de surintensité (courant) ou de surchauffe (température) avec un horodatage précis.
- **Gérer le parc automobile** : Conserver les données relatives aux véhicules de l'utilisateur (ID, nom, kilométrage) et appliquer instantanément les demandes d'ajout, de modification ou de suppression.
- **Assurer la persistance et l'indépendance des données** : Garantir la sauvegarde et la sécurité des informations, qui restent consultables et fiables même en cas de redémarrage ou de coupure du serveur Qt.

Bootstrap



Bootstrap 5 est utilisé comme framework CSS pour concevoir et structurer l'interface utilisateur graphique (IHM) de l'application web de supervision. Intégré directement dans les en-têtes des fichiers de présentation (tels que `borne.php` et `mesures.php`), il fournit un ensemble de règles CSS et de composants pré-stylisés. Son utilisation évite de réécrire des feuilles de style complexes à la main tout en standardisant l'identité visuelle de l'application.

- **Mise en page réactive (Grille Responsive et conteneurs) :** Le framework s'appuie sur un système de grille flexible basé sur Flexbox. En utilisant des classes structurales comme `.container-fluid`, `.row` ou `.col-md`, l'IHM s'adapte automatiquement à la résolution de l'écran (ordinateurs, tablettes ou smartphones). Cela garantit que les cartes de filtrage et le graphique restent lisibles quel que soit le terminal utilisé.
- **Stylisation et intégration des composants dynamiques :** Bootstrap 5 fournit des classes prêtes à l'emploi pour uniformiser les éléments interactifs. Par exemple, les boutons de validation exploitent des classes spécifiques comme `.btn-afficher`, et les barres de navigation tirent parti de la structure `.navbar` et `.nav-item`. De plus, le framework s'associe nativement avec des plugins tiers comme DataTables (via la feuille `dataTables.bootstrap5.min.css`) pour habiller le tableau `#table_mesures` et lui donner un aspect moderne et épuré.
- **Séparation du design et de la structure HTML :** Grâce à l'utilisation des classes utilitaires intégrées (gestion des marges, alignement du texte avec `.text-center`), le code HTML de la page reste propre, lisible et allégé. L'apparence visuelle globale est entièrement gérée par les fichiers de style du framework, ce qui simplifie grandement la maintenance graphique de l'application.

DataTables

L'extension jQuery DataTables est utilisée pour optimiser la gestion et l'affichage des données tabulaires au sein de l'IHM. Appliquée directement sur le tableau HTML affichant les données historiques (comme dans la page `mesures.php`), elle transforme une simple liste d'enregistrements statiques en une interface de consultation dynamique et interactive. Ce composant est indispensable pour manipuler efficacement les gros volumes de mesures stockés sur le SSD du Raspberry Pi 4.

- **Pagination automatique et réduction de la surcharge d'affichage :** Lorsque le tableau contient des centaines de lignes (historique des mesures de charge), DataTables segmente l'affichage via une pagination fluide. En configurant le paramètre de sélection du nombre de lignes, le système limite la surcharge cognitive pour l'utilisateur et allège considérablement le rendu visuel du navigateur, évitant ainsi le gel ou le ralentissement de l'interface graphique.
- **Moteur de recherche et de tri en temps réel :** DataTables intègre nativement un champ de recherche global qui filtre instantanément les lignes du tableau au fur et à mesure de la saisie de l'utilisateur, sans nécessiter de nouvelle requête SQL vers le serveur PHP. De plus, il ajoute des contrôles de tri interactifs (ascendant / descendant) sur les en-têtes de colonnes, permettant par exemple de classer rapidement les données par ordre chronologique ou par puissance mesurée.
- **Intégration transparente avec le design (Bootstrap 5) :** Grâce à sa feuille de style dédiée à Bootstrap 5 (`dataTables.bootstrap5.min.css`) et à son script d'adaptation (`dataTables.bootstrap5.min.js`), DataTables hérite automatiquement des classes graphiques du framework. Le tableau (`#table_mesures`) conserve ainsi un design uniforme, épuré et parfaitement réactif (responsive), en accord avec le reste de l'application.

Highcharts



La bibliothèque JavaScript Highcharts est utilisée pour générer et afficher le graphique interactif représentant l'évolution de la puissance de charge d'une borne sur la page `borne.php`. Alimentée dynamiquement en données par les flux JSON du

contrôleur PHP, elle permet de traduire visuellement des données numériques brutes (provenant de la table MESURES) en courbes ou en diagrammes clairs et immédiatement interprétables par l'utilisateur.

- **Rendu dynamique et interactif de la puissance (IHM) :** Highcharts prend en charge la construction complète du composant graphique au sein d'un conteneur HTML dédié (`<div id="container"></div>`). Grâce à son API, elle permet d'associer un axe temporel (les horodatages) à un axe de valeurs (la puissance en kW). L'interface offre des fonctionnalités interactives natives, telles que le survol des points à la souris pour afficher des info-bulles de précision (*tooltips*) ou le masquage dynamique de séries de données.
- **Extraction et sauvegarde via le module exporting :** L'intégration du script additionnel `exporting.js` ajoute un bouton de menu contextuel (représenté par trois lignes horizontales) en haut à droite du graphique. Ce module gère l'exportation visuelle du graphique directement côté client : l'utilisateur peut télécharger instantanément le rendu graphique au format image vectorielle ou matricielle (PNG, JPEG, PDF, SVG) pour l'intégrer dans des rapports externes.
- **Conversion et téléchargement des données brutes via export-data :** Couplé au module d'exportation, le script `export-data.js` permet d'extraire la structure de données sous-jacente qui a servi à tracer le graphique. Sans faire de nouvelle requête vers la base de données MariaDB, ce module convertit le jeu de données local du graphique en fichiers tableurs standardisés (CSV ou XLSX) ou en tableau HTML, offrant ainsi à l'administrateur une autonomie complète pour analyser les données de consommation.

Extension PDO



PDO (PHP Data Objects) est utilisée comme interface

d'abstraction pour l'accès à la base de données MariaDB installée sur le Raspberry Pi 4. Intégrée au sein des fichiers modèles (comme `modele_mesures.inc.php`), PDO permet d'exécuter des requêtes relationnelles structurées. Elle dissocie complètement la logique de programmation PHP

des spécificités techniques du moteur de base de données, garantissant ainsi la sécurité et la portabilité des accès aux données de charge.

- **Sécurisation contre les injections SQL (Requêtes préparées) :** L'interface PDO est indispensable pour neutraliser les failles de sécurité lors de la manipulation de variables dynamiques. Dans la fonction `GetDernieresMesuresParBorne`, PDO utilise l'instruction `$connexion->prepare()` pour compiler la structure de la requête SQL en amont, puis associe l'identifiant de la borne de manière hermétique grâce à `$requete->bindParam(":idBorne", $idBorne)`. Ce mécanisme empêche toute altération malveillante de la requête par injection de code SQL externe.
- **Typage et mapping automatique des résultats :** PDO facilite la récupération des enregistrements sous forme de structures exploitables par le code PHP. Grâce à l'utilisation de la méthode `$requete->fetch(PDO::FETCH_ASSOC)`, PDO extrait chaque ligne de la table `MESURES` sous la forme d'un tableau associatif indexé par le nom des colonnes SQL (`id_mesure`, `horodatage`, `puissance`, `id_session`). Cela permet de reconstruire proprement un tableau de données qui sera converti pour l'IHM.
- **Gestion centralisée de la connexion et des erreurs (PDOException) :** L'ensemble des appels PDO est encapsulé dans des blocs d'interception `try { ... } catch (PDOException $erreur)`. En cas de déconnexion physique du stockage SSD ou d'arrêt du service MariaDB, l'exception est immédiatement capturée, ce qui permet d'afficher un message d'erreur contrôlé via `$erreur->getMessage()` et d'interrompre proprement l'exécution du script via `die()`, évitant ainsi l'affichage de détails sensibles sur l'infrastructure du serveur.

8 Diagramme de classes

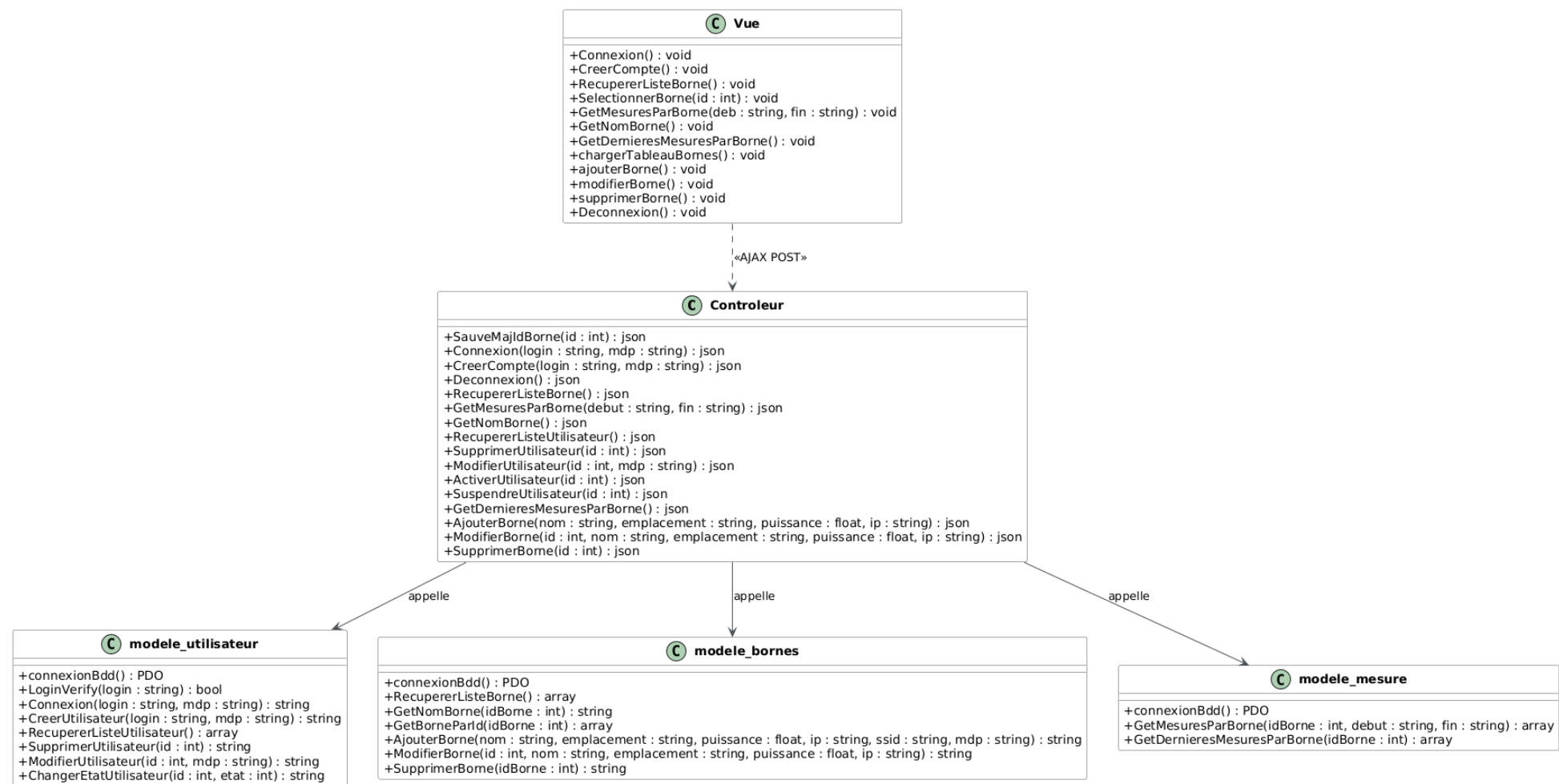


Figure 16: Diagramme de classes

Vue (le front-end JS)

C'est tout ce que l'utilisateur déclenche depuis le navigateur. Chaque méthode envoie une requête **AJAX POST** au contrôleur.

- **Connexion()** → envoie login + mdp pour s'authentifier
- **CreerCompte()** → envoie login + mdp pour créer un compte
- **RecupererListeBorne()** → demande la liste des bornes (page choix_bornes et gerer_bornes)
- **SelectionnerBorne(id)** → sauvegarde l'id de la borne choisie en session
- **GetMesuresParBorne(deb, fin)** → demande les mesures sur une période → alimente le graphique Highcharts
- **GetNomBorne()** → récupère le nom de la borne en session pour l'afficher dans la navbar
- **GetDernieresMessuresParBorne()** → demande les 30 dernières mesures → alimente le tableau DataTables
- **chargerTableauBornes()** → charge le tableau DataTables de la page admin des bornes
- **ajouterBorne()** → envoie les données du formulaire pour créer une borne
- **modifierBorne()** → envoie les données modifiées d'une borne
- **supprimerBorne()** → envoie l'id de la borne à supprimer
- **Deconnexion()** → détruit la session et redirige vers login.php

Controleur (controleur.php)

C'est le **chef d'orchestre**. Il reçoit toutes les requêtes AJAX, lit le paramètre **commande**, appelle la bonne fonction de modèle et retourne toujours du JSON.

- **SauveMajIdBorne(id)** → stocke l'id borne dans \$_SESSION['idBorne']
- **Connexion(login, mdp)** → délègue à modele_utilisateur, retourne {statut: ok/erreur}
- **CreerCompte(login, mdp)** → délègue à modele_utilisateur, retourne {statut: ok/existe/erreur}
- **Deconnexion()** → détruit \$_SESSION, supprime le cookie de session
- **RecupererListeBorne()** → délègue à modele_bornes, retourne le tableau JSON des bornes
- **GetMesuresParBorne(debut, fin)** → lit \$_SESSION['idBorne'], calcule 30 jours par défaut si pas de dates
- **GetNomBorne()** → lit \$_SESSION['idBorne'], retourne le nom en JSON
- **RecupererListeUtilisateur()** → retourne tous les utilisateurs (page admin)
- **SupprimerUtilisateur(id)** → supprime et retourne le statut
- **ModifierUtilisateur(id, mdp)** → met à jour le mot de passe
- **ActiverUtilisateur(id)** → appelle ChangerEtat avec etat=1
- **SuspendreUtilisateur(id)** → appelle ChangerEtat avec etat=0

- **GetDernieresMesuresParBorne()** → lit `$_SESSION[ 'idBorne' ]`, retourne les 30 dernières mesures
- **AjouterBorne(...)** → crée une borne (ssid et mdp_usine codés en dur)
- **ModifierBorne(id, ...)** → met à jour les infos d'une borne
- **SupprimerBorne(id)** → supprime une borne

modele_utilisateur

Gère tout ce qui touche à la table **UTILISATEURS** en base.

- **connexionBdd()** → ouvre la connexion PDO MySQL
- **LoginVerify(login)** → vérifie si le login existe ET est actif (etat=true) → retourne un booléen
- **Connexion(login, mdp)** → vérifie le login puis compare le mdp avec `password_verify()` → initialise la session si OK
- **CreerUtilisateur(login, mdp)** → vérifie que le login n'existe pas, hache le mdp avec `password_hash()` et insère
- **RecupererListeUtilisateur()** → SELECT de tous les utilisateurs (id, login, etat)
- **SupprimerUtilisateur(id)** → DELETE par id
- **ModifierUtilisateur(id, mdp)** → UPDATE du mot de passe (re-haché)
- **ChangerEtatUtilisateur(id, etat)** → UPDATE du champ etat (0 ou 1)

modele_bornes

Gère tout ce qui touche à la table **BORNES**.

- **connexionBdd()** → ouvre la connexion PDO
- **RecupererListeBorne()** → SELECT de toutes les bornes (id, nom, emplacement, puissance, ip_usine)
- **GetNomBorne(idBorne)** → SELECT du nom d'une borne par son id
- **GetBorneParId(idBorne)** → SELECT * d'une borne par son id (tous les champs)
- **AjouterBorne(nom, emplacement, puissance, ip, ssid, mdp)** → INSERT d'une nouvelle borne
- **ModifierBorne(id, nom, emplacement, puissance, ip)** → UPDATE des infos d'une borne
- **SupprimerBorne(idBorne)** → DELETE d'une borne (attention : cascade possible sur SESSIONS/MESURES)

modele_mesure

Gère la table **MESURES** en jointure avec **SESSIONS**.

- **connexionBdd()** → ouvre la connexion PDO
- **GetMesuresParBorne(idBorne, debut, fin)** → jointure MESURES ↔ SESSIONS filtrée par borne et période → retourne `{categories: [...], series: [...]}` prêt pour Highcharts
- **GetDernieresMesuresParBorne(idBorne)** → même jointure, triée par date DESC, limitée à **30 lignes** → retourne un tableau pour DataTables

9 GitHub

GitHub a été l'outil de gestion de version choisi pour ce projet. Concrètement, chaque membre du groupe travaillait sur sa propre branche correspondant à sa partie du projet (ESP32, application mobile, interface web), puis intégrait ses modifications au code principal une fois validées. Cette organisation nous a permis de travailler en parallèle sans risque d'écraser le travail des autres.

Au cours du développement, cet outil s'est révélé particulièrement utile lors des phases de débogage. En cas de régression, il suffisait de consulter l'historique des commits pour identifier la modification à l'origine du problème et revenir à une version fonctionnelle antérieure. Cette traçabilité a été un atout majeur dans la gestion des crashes rencontrés sur l'ESP32.

Le dépôt GitHub du projet est accessible à l'adresse suivante :

https://github.com/Corentinh2/PROJET_GESTIONNAIRE_DE_CHARGE/tree/Andrea



10 Fiche de test

<u>Référence du module testé :</u>		F4.1°ETu.1°Test
Date du test :	30/04/2026	
Type du test :	Test fonctionnel	
Objectif du test :	Vérifier le parcours complet : connexion admin, sélection de la borne n°1, récupération des données de puissance depuis MariaDB et affichage dynamique du graphique Highcharts.	
<u>Conditions du test</u>		

État initial		Environnement du test
La raspberry Pi est branché et bien allumé. Avoir NetBeans d'ouvert.		Navigateur Web (Firefox). Connexion internet requise pour charger la bibliothèque highcharts.js
Procédures du test		
N°	Opérations	Résultats attendus
1	Ouvrir un terminal et taper cette commande « sshfs pi@172.18.59.133:/var/www/html/gestionnaire_de_charge/gestionnaireCharge ~/remote ».	Si la connexion s'établit il n'y a pas de message d'erreur.
2	Ouvrir le projet « remote » et lancer le projet, saisir le login "admin" et le mot de passe "admin1234", puis cliquer sur SE CONNECTER.	L'authentification réussit. L'utilisateur est redirigé vers la page GestionUtilisateurs.
3	Cliquer sur "Retour aux bornes" dans la barre latérale pour accéder à choix_borne.php.	La liste des bornes disponibles s'affiche. La borne n°1 est visible avec son nom, son emplacement et sa puissance. Le bouton "Supprimer" est visible car l'utilisateur est admin.
4	Cliquer sur la carte de la Borne n°1.	Un graphique s'affiche en barre graphe.
5	Vérifier la carte "Puissance Actuelle".	La valeur affichée correspond à la dernière entrée de la table MESURE pour la borne n°1 (méthode GetDerniereMesure). L'horodatage est correct.
6	Changer la plage de temps en cliquant sur "Jour", "Semaine" puis "Mois".	Le graphique Highcharts se met à jour. Les données traitées par GetMesuresParBorne sont correctement transmises au JavaScript via json_encode.
7	Cliquer sur "Historique MariaDB" dans la barre latérale.	La page historique.php affiche une liste tabulaire des 50 dernières mesures brutes pour la borne n°1.
8	Cliquer sur "Déconnexion" dans la barre latérale.	La session est détruite. L'utilisateur est redirigé vers la page login.php.

11 Diagramme de Gantt

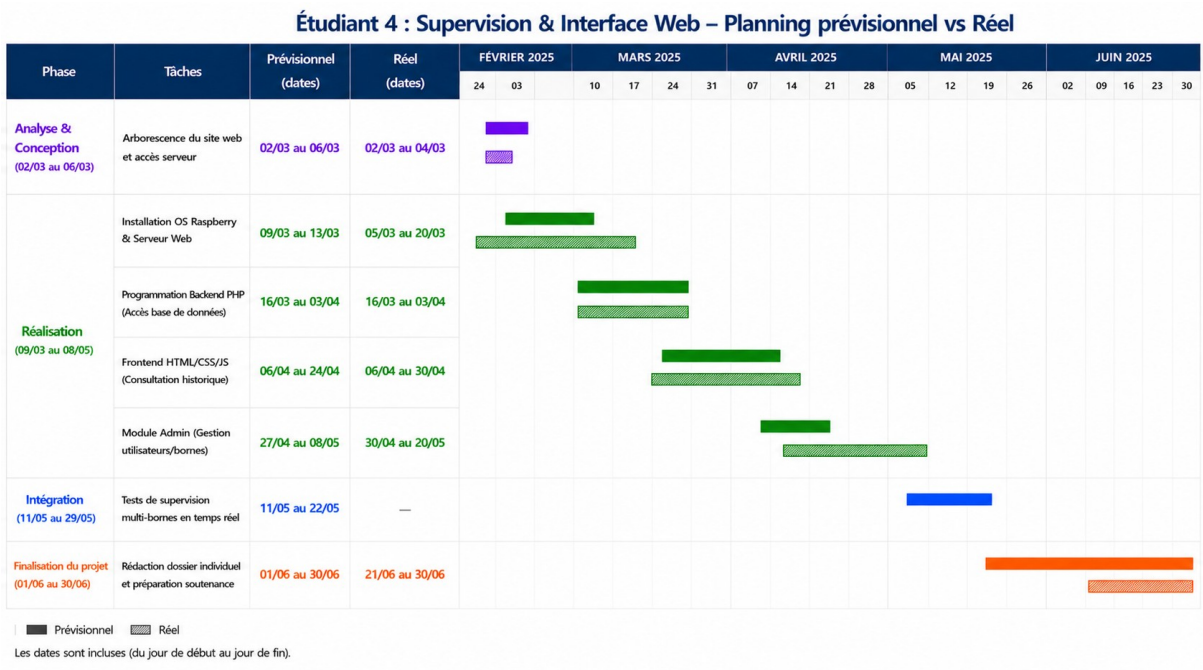


Figure 17: Diagramme de Gantt

12 Compétences acquises

L'intégration et la maîtrise de cet écosystème technologique complet (PHP 8, MariaDB, PDO, AJAX, Bootstrap 5, DataTables et Highcharts) ont permis de développer de solides compétences d'ingénierie logicielle, structurées autour de trois grands axes :

1. Maîtrise des Architectures Logicielles et du Développement Back-End

- **Structuration et modularité (Pattern MVC 2) :** La mise en œuvre du motif Modèle-Vue-Contrôleur a développé ma capacité à concevoir des applications web hautement structurées. Séparer de manière étanche la logique métier (Modèle PHP), les mécanismes d'aiguillage (Contrôleur) et l'interface utilisateur (Vue) m'a permis d'assimiler les bonnes pratiques de développement industriel, garantissant la maintenabilité et l'évolutivité d'un code source.
- **Sécurisation des données et des accès :** L'utilisation conjointe de PDO pour contrer les injections SQL (via les requêtes préparées et le *binding* de variables) et des fonctions de hachage natives (`password_hash`) m'a sensibilisé aux problématiques majeures de la cybersécurité applicative. De plus, la gestion rigoureuse des sessions PHP a renforcé mes compétences dans le contrôle d'accès et la protection des données sensibles.

2. Optimisation des Interfaces Évoluéées (Front-End) et UX

- **Développement asynchrone et performance :** L'implémentation des requêtes AJAX (via jQuery) connectées à des flux de données JSON a transformé ma vision de l'interaction homme-machine. J'ai appris à concevoir des interfaces fluides et dynamiques qui s'affranchissent des rechargements de page, optimisant ainsi l'expérience utilisateur tout en économisant les ressources processeur du serveur (Raspberry Pi 4).
- **Intégration de composants graphiques tiers :** L'exploitation des bibliothèques Highcharts et DataTables m'a permis d'acquérir une expertise dans la manipulation, la pagination et la valorisation visuelle de gros volumes de données. J'ai appris à configurer des modules complexes (systèmes d'exportation de données, tris en temps réel) pour offrir des outils d'analyse puissants à l'utilisateur final.
- **Design adaptatif (Responsive Design) :** À travers l'utilisation du framework Bootstrap 5, j'ai maîtrisé les concepts de mise en page moderne (système de grille, Flexbox) permettant de garantir une compatibilité visuelle absolue sur tout type de terminal (PC, tablette, smartphone).

3. Autonomie et Rigueur Méthodologique

- **Résolution de problèmes et Débogage :** Le développement complet de cette chaîne de liaison (du stockage SSD à l'affichage sur le navigateur) a aiguisé mon esprit d'analyse face

aux pannes. J'ai appris à isoler les anomalies à chaque niveau de l'architecture (requête SQL défaillante dans le modèle, erreur de routage dans le contrôleur ou mauvaise interprétation du JSON côté client).

- **Adaptation aux contraintes matérielles :** Travailler directement sur un environnement embarqué (serveur local sur Raspberry Pi) m'a forcé à prendre en compte l'optimisation des performances logicielles afin de préserver le matériel et fluidifier les échanges de données.

13 Perspectives d'améliorations

Actualisation automatique du graphique sans recharger (Temps Réel)

Actuellement, pour voir les nouvelles mesures de consommation d'une borne, l'utilisateur doit appliquer un filtre ou rafraîchir manuellement ses données. Une évolution intéressante serait d'ajouter un rafraîchissement automatique (par exemple toutes les 5 secondes). Le graphique de consommation se mettrait ainsi à jour tout seul sous les yeux de l'utilisateur, permettant de suivre la charge d'un véhicule en direct.

2. Création d'un tableau de bord général (Dashboard)

L'application se concentre aujourd'hui sur la consultation d'une borne à la fois. Une excellente perspective serait de créer une page d'accueil globale présentant un tableau de bord. Celui-ci afficherait des indicateurs clés en un coup d'œil : le nombre total de bornes en fonctionnement, le nombre de recharges en cours, la consommation totale de la journée et des alertes visuelles si une borne est en panne.

3. Système de gestion des utilisateurs et des droits (Profils)

Pour l'instant, le système distingue principalement les administrateurs des utilisateurs classiques. On pourrait faire évoluer l'application en créant des profils plus précis. Par exemple : un profil "Technicien" qui peut modifier la configuration des bornes, un profil "Client" qui peut seulement voir l'historique de ses propres recharges, et un profil "Superviseur" ayant une vue d'ensemble sur tout le réseau.